

Architecture et Programmation Assembleur - Contrôle Continu

 Classe: B2B Durée: 2h Semestre 2 (2025-2026) Campus Keyce

Exercice 1 : Fondamentaux de la Machine

4
points

1.1. Conversions (2 points)

a) 10110101_2 en décimal✓ 181_{10}

$$\begin{aligned} &1 \times 128 + 0 \times 64 + 1 \times 32 + 1 \times 16 + 0 \times 8 \\ &+ 1 \times 4 + 0 \times 2 + 1 \times 1 = 181 \end{aligned}$$

b) $3F_{16}$ en binaire✓ $0011\ 1111_2$

$$3 = 0011 ; F = 1111$$

c) 127_{10} en binaire sur 8 bits✓ $0111\ 1111_2$

C'est la valeur maximale positive signée sur 8 bits.

d) -42_{10} en complément à 2 (8 bits)✓ $1101\ 0110_2$

$$\begin{aligned} &+42 = 0010\ 1010 \\ &\text{C. à 1} = 1101\ 0101 \\ &+ 1 = 1101\ 0110 \end{aligned}$$

1.2. Opérations et débordement (2 points)

```
  1101 0110
+ 0010 1101
-----
1 0000 0011
```

i. Résultat en binaire et décimal (non signé)

→ Binaire : 0000 0011

→ Décimal : 3

L'addition donne 259 (sur 9 bits).
Sur 8 bits, le résultat conservé est
3 (modulo 256).

ii. Y a-t-il une retenue (Carry) ?

✓ Oui (CF = 1)

Une retenue est générée hors du
bit de poids fort car $214 + 45 =$
 $259 > 255$.

iii. Y a-t-il un débordement (Overflow en signé) ?

✗ Non (OF = 0)

L'opération correspond à $(-42) +$
 $(+45) = +3$.
L'addition d'un nombre positif et
d'un nombre négatif ne peut
jamais produire de débordement
de capacité (Overflow).



Exercice 2 : Architecture du Processeur

4
points

2.1. Association des Registres (2 points)

EIP

Adresse de la
prochaine
instruction

ESP

Pointeur de pile

EFLAGS

Indicateurs d'état

EAX

Accumulateur pour
résultats

2.2. Rôles des registres (2 points)

- Le registre **EIP** contient l'adresse de la prochaine instruction à exécuter.
- Le registre **ECX** est utilisé comme compteur dans les boucles.
- Le registre **ESP** pointe vers le sommet de la pile.
- Le registre **EFLAGS** stocke les indicateurs (flags).

Note : Bien que non listé à la question 2.1, ECX est le registre spécifiquement désigné pour servir de compteur de boucle (Counter Register) en architecture x86.



Exercice 3 : Jeu d'Instructions

5 points

3.1. Modes d'adressage (2 points)

#	Instruction	Mode d'adressage (Opérande Source)
1	MOV EAX, 100	Immédiat (La valeur est dans l'instruction)

#	Instruction	Mode d'adressage (Opérande Source)
2	<code>MOV EAX, [100]</code>	Direct / Absolu (L'adresse mémoire est explicite)
3	<code>MOV EAX, EBX</code>	Registre (La valeur est lue dans un registre)
4	<code>MOV EAX, [EBX]</code>	Indirect par registre (L'adresse mémoire est ciblée par EBX)

3.2. Traduction d'expressions (3 points)

Expression à coder : `resultat = (a * 2) + (b / 4) - c`

```
; Utilisation de SHL (x2) et SHR (/4)
MOV EAX, [a]           ; On charge la variable 'a' dans eax
SHL EAX, 1             ; Décalage à gauche de 1 bit = a * 2

MOV EBX, [b]           ; On charge la variable 'b' dans ebx
SHR EBX, 2             ; Décalage à droite de 2 bits = b / 4

ADD EAX, EBX           ; On additionne : (a * 2) + (b / 4)
SUB EAX, [c]           ; On soustrait 'c'

MOV [resultat], EAX ; On stocke le résultat en mémoire
```



Exercice 4 : Programmation sous SASM

7
points

Écriture d'un programme complet (Calcul de la somme et de la moyenne d'un tableau).

```
%include "io.inc"
```

```
section .data
```

```
; 1. Tableau de 8 entiers et taille du tableau (1,5 pts)
```

```
tableau dd 10, 20, 30, 40, 50, 60, 70, 80
```

```
taille dd 8
```

```
section .bss
```

```
; Variable non initialisée pour la moyenne
```

```
moyenne resd 1
```

```
section .text
```

```
global CMAIN
```

```
CMAIN:
```

```
mov ebp, esp; pour le debug correct dans SASM
```

```
; 2. Calcul de la somme (2 pts)
```

```
mov ecx, [taille] ; Initialisation du compteur de boucle (ecx =
```

```
xor eax, eax ; eax sera notre somme cumulée (eax = 0)
```

```
mov esi, tableau ; esi pointe sur le début du tableau
```

```
boucle:
```

```
add eax, [esi] ; Ajout de l'élément pointé par esi à la somme
```

```
add esi, 4 ; Déplacement au prochain entier (32 bits = 4
```

```
loop boucle ; Décrémente ecx, saute à 'boucle' si ecx != 0
```

```
; Sauvegarde de la somme dans ebx pour l'affichage ultérieur
```

```
mov ebx, eax
```

```
; 3. Calcul de la moyenne (2 pts)
```

```
; La division 'div' s'opère sur edx:eax par l'opérande
```

```
xor edx, edx ; Mise à 0 de edx (extension de eax)
```

```
mov ecx, [taille] ; On recharge la taille comme diviseur
```

```
div ecx ; eax = eax / ecx, edx = reste
```

```
mov [moyenne], eax ; Stockage du résultat dans la variable .bss
```

```
; 4. Affichage à l'écran (1,5 pts)
```

```
PRINT_DEC 4, ebx ; Affiche la somme stockée, par exemple: 360
```

```
NEWLINE
```

```
PRINT_DEC 4, [moyenne] ; Affiche la moyenne stockée, par exemple: 45
```

```
NEWLINE
```

```
xor eax, eax  
ret
```

```
; Code de retour 0
```